第1页 | 开场与选题

老师、同学们好，我这次选择的是 Mandol 智能体记忆系统，具体方向是超长记忆增强。经过前期阅读，我把任务暂时缩小到一个比较具体的问题，就是 L1 摘要中的近重复记忆合并。

这次主要汇报三个方面：我对原系统的理解，准备改进的地方，以及目前已经完成的验证。现在还是初步阶段，完整功能还没有全部做完。下面先用一个例子说明为什么考虑这个问题。

第2页 | 为什么想改进记忆合并

这一页的三条记录都提到了去上海，但它们并不完全一样。A只说周五去上海参加会议，B还补充了周日返回。如果因为两条内容很像就直接保留A，返回日期就丢掉了。

C的情况又不一样，它有一个“如果请假获批”的条件，不能直接当成已经确定的行程。所以我想解决的，不只是判断两条文字像不像，还包括判断合并以后有没有少掉信息、改变原来的意思。

我的初步目标是减少重复内容，同时保留新增细节、条件和原始来源。这里用的是解释需求的例子，并不是实验结果。接下来介绍一下这个问题在 Mandol 中处于什么位置。

第3页 | 我对 Mandol 的基本理解

我把 Mandol 理解为智能体的记忆管理部分。它负责保存信息，把原始内容加工成不同形式的记忆，再在需要时把相关内容找出来。外部应用拿到这些内容之后，再交给回答模型生成回复。

它的高阶记忆不只有摘要，还包括情景记忆和实体关系记忆。我目前主要关注分层摘要这一部分，准备改的是 L1 摘要生成之后的去重处理。这样任务范围会比较明确，也能复用原来的存储和检索功能。下面是阅读源码时整理出的几个核心对象。

第4页 | 读代码时先关注这几个对象

MemoryUnit表示一条记忆，里面有内容、唯一标识、元数据和向量。MemorySpace主要记录哪些记忆属于哪个空间，以及子空间之间的关系，不是每个空间都再复制一份完整内容。

SemanticMap负责管理内容和检索索引。SemanticGraph在它的基础上管理图节点和关系，并协调相关操作。因此图里SemanticGraph指向SemanticMap，表示它持有并使用这个对象，不是继承关系。

这个分工也影响后面的改进：不能只考虑文本有没有合并，还要考虑来源、空间和索引是否一致。为了先控制实现难度，我准备优先在记忆正式写入前进行处理。

第5页 | 准备改在原流程的哪个位置

沿着原代码看，相关流程可以概括为：先有L0基础记忆，再按会话提取L1摘要，经过可选的去重之后，继续聚合成L2，最后写入系统。

我准备优先改中间的L1去重位置。这个时候处理的是构建结果列表，还没有对已有历史图做节点迁移，比较适合作为第一版。整理后的L1仍然交给原来的L2和写入流程，尽量减少对其他模块的影响。

另外，保存一条MemoryUnit本身，不代表所有高阶记忆都已经构建完成。这几个步骤在代码里是需要分开看的。接下来具体看原来的L1去重怎样处理。

第6页 | 原来的去重代码是怎样处理的

原方法会先按摘要类型分组，把每条摘要的前500个字符交给模型判断。模型返回合并组以后，这里这行代码会把组里第一个ID加入保留集合，最后留下这个ID对应的原结果。

在这段方法里，并没有使用模型返回的融合文本，也没有把其他条目的来源列表一起汇总。因此，我关注两个可能的问题：第一项不一定包含全部补充信息；保留这一项以后，也可能只剩它自己的来源。

这里需要区分一下，源码能确认的是处理方式，不能直接据此断言真实模型经常导致信息丢失。它为改进提供了一个具体切入点，实际效果还要另外验证。

第7页 | 哪些内容可以合并，哪些先保留

第一版准备采取比较保守的方式。同一个事实的重复表达，可以保留一条并汇总来源；如果一条包含另一条的信息，也要先检查主体、时间和条件是不是一致，再决定留下哪一条。

如果数字、否定、时间或条件不同，就先保留区别。例如“喜欢辣”和“不喜欢辣”虽然文字相近，含义却不同。遇到无法判断或者模型返回格式异常的情况，也先不合并，并记录原因。

这会减少一些压缩机会，但目前我更希望先把“不该丢的信息不要丢”这件事做好。现在的简单规则还不能可靠处理所有条件句，这也是后面需要完善的地方。

第8页 | 目前考虑的处理步骤

根据这些要求，我先把处理过程分成几项职责：组织候选、判断关系、生成合并计划，以及校验后接回原流程。图上的几个名字是拟定的对象职责，还没有全部实现成独立的类。

合并计划主要记录哪些条目被合并、留下哪条内容、汇总哪些来源，以及为什么这样处理。第一版先选用已有文本，暂时不让模型重新写摘要，避免同时引入新的生成错误。

在接入检查中还发现，原L1结果没有自动带上当前原型需要的user_id。所以正式接入时，要从调用方或可靠来源传递主体信息，不能把样本编号或者相同参与者直接当成同一个事实主体。下面用一个例子说明期望的输出。

第9页｜一个希望实现的合并例子

这里A只有“周五去上海”，B还补充“周日返回”。确认两者主体和语境一致，而且B确实包含A的信息后，就可以选择B作为保留下来的内容。

同时，来源需要保存为a和b，并记录这次合并包含A、B两个成员。这样后面想检查这条记忆是怎么来的，还能找到对应记录。

这里说的来源保留，是来源ID层面的追溯，还没有细化到每句话对应哪一段证据。这个例子表达的是希望实现的行为，不代表所有相似文本都能这样处理。接下来介绍已经做过的链路验证。

第10页｜目前做过哪些检查

目前已经把合并结果通过原来的写入适配和写入队列，放进真实的SemanticGraph中，检查了来源元数据和空间归属。使用FAISS和BM25查询时，也能找到保留下来的补充信息。

还检查了RocksDB的冷数据换出、按需加载和删除。保存之后关闭图资源，再由一个新的进程重新加载，对比内容、来源、关系、向量、空间和检索结果，两种存储模式下都通过了一致性检查。

不过，这里必须说明，嵌入使用的是确定性测试向量，不是真实语义模型。因此，这一步证明的是图、索引和磁盘之间的链路能够工作，还不能说明合并判断更准确，或者最终问答更好。测试里的来源关系边也是显式添加的，不代表合并组件已经自动完成了关系维护。

第11页｜目前的进度和接下来要做的事

最后总结一下，目前已经完成了主要源码阅读、对象和流程梳理，形成了独立原型，并验证了真实图、检索索引和保存恢复对这些数据的支持。

接下来，首先要补齐主体信息的传递，完善条件和时间差异的判断，然后把它接入完整构建流程。之后再用人工复核过的新数据，比较信息有没有丢失、来源是否完整，以及真实模型下的合并和问答效果。

所以目前比较合适的结论是：这个方向有明确的接入位置，相关存储链路也能支持合并后的结果，但整个改进还没有全部完成，还需要继续验证。我的汇报就到这里，谢谢老师和同学们。